CMS Dummy Data File Structure

This document describes the binary files generated by `main.c`. It is intended for collaborators who need to read or write the generated data.

Important format characteristics

- Files are **binary**, not text or JSON.
- Records are written directly with C `fwrite()`:

  ```
  fwrite(&record, sizeof(record), 1, fp);
  ```

- The format uses the compiler's native struct layout, including alignment and padding bytes.
- The current files were generated with a 64-bit MinGW GCC build on Windows. The current layout assumes:
  - `char` = 1 byte
  - `int` = 4 bytes
  - `uint32_t` = 4 bytes
  - `float` = 4 bytes, IEEE-754 representation
  - `UserType` enum = 4 bytes
  - little-endian byte order
- Do not assume that a different compiler, architecture, or compiler packing option will produce the same offsets.
- Strings are fixed-size character arrays. They are normally NUL-terminated and the unused bytes are zero-filled because the generator initializes structs with `{0}`. A string that exactly fills an array may not have a terminating NUL.
- There is no magic number, version field, checksum, or schema identifier. Readers must already know which file type they are opening.
- All offsets in this document are byte offsets from the beginning of the relevant file or struct and are zero-based.

Files generated

File pattern	Format	Current size	Description
`credentials.dat`	repeated `User` records	504 bytes	21 login records
`NNNN.notice`	one `Notice` record	632 bytes	One notice per file
`STUDENT_ID.student`	header plus four data records	1870 bytes	One complete student record

File pattern	Format	Current size	Description
`TEACHER_ID.teacher`	one `TeacherProfile` record	415 bytes	One complete teacher record

The generator currently creates:

- 10 student credential records
- 10 teacher credential records
- 1 administrator credential record
- 10 student files
- 10 teacher files
- 10 notice files

Common types

`UserType`

```
typedef enum {
    USER_TYPE_TEACHER = 1,
    USER_TYPE_STUDENT = 2,
    USER_TYPE_ADMIN = 3
} UserType;
```

The enum is stored as a 4-byte integer in the current build.

Value	Meaning
1	Teacher
2	Student
3	Administrator

`Date`

```
typedef struct {
    int day;
    int month;
    int year;
} Date;
```

Current size: **12 bytes**.

Offset	Size	Field	Type
0	4	`day`	`int`
4	4	`month`	`int`

Offset	Size	Field	Type
8	4	`year`	`int`

The generator uses the calendar interpretation `day-month-year`, despite the comment in the source using the abbreviation `dd-mm-yyyy`.

`credentials.dat`

Record layout

`credentials.dat` is a sequence of 21 `User` records. Each record is **24 bytes**, so the file size is currently `21 * 24 = 504` bytes.

```
typedef struct {
    char id[9];
    char pass[9];
    UserType userType;
} User;
```

Offset	Size	Field	Notes
0	9	`id`	Up to 8 characters plus NUL
9	9	`pass`	Up to 8 characters plus NUL
18	2	padding	Alignment before `userType`
20	4	`userType`	Values 1, 2, or 3

To read record number `n` (zero-based), seek to:

```
record_offset = n * 24
```

Records are written in this order:

1. Students `52501001` through `52510010`
2. Teachers `10000001` through `10000010`
3. Administrator `00000000`

The first student and first teacher use password `00000000`. Other generated passwords are eight-character numeric strings and are regenerated randomly each time the generator runs.

Teacher files

Each `TEACHER_ID.teacher` file contains exactly one `TeacherProfile` record. Teacher IDs use the same eight-digit format as student IDs. The file name must match the `teacherId` field: for example, `10000001.teacher` contains the record whose `teacherId` is `10000001`. The generated email address follows `last_name@dept_short_code.kuet.ac.bd`, and phone numbers use Bangladesh's `01XXXXXXXXX` format.

```
typedef struct {
    char name[80];
    char teacherId[9];
    char designation[20];
    char department[50];
    char phone[16];
    char email[80];
    char researchSpecialization[160];
} TeacherProfile;
```

The current record size is **415 bytes**. All fields are fixed-size, NUL-terminated character arrays and have no internal alignment padding.

Struct offset	Size	Field
0	80	`name`
80	9	`teacherId`
89	20	`designation`
109	50	`department`
159	16	`phone`
175	80	`email`
255	160	`researchSpecialization`

Notice files

Each file named `NNNN.notice` contains exactly one `Notice` struct. For example, `0001.notice` contains notice ID `0001`.

```
typedef struct {
    char noticeId[5];
    Date date;
    char Title[100];
    char Description[512];
} Notice;
```

Current size: **632 bytes**.

Offset	Size	Field	Notes
0	5	`noticeId`	Four-digit ID plus NUL
5	3	padding	Alignment before `Date`
8	12	`date`	`Date` struct
20	100	`Title`	Full notice title
120	512	`Description`	Full notice description

There is no collection file for notices. To read all notices, enumerate the `*.notice` files and read one `Notice` record from each file.

Student files

Each `STUDENT_ID.student` file contains five records in a fixed sequence:

```
StudentFileHeader
StudentProfile
AcademicRecord
HallDetails
LibraryDetails
```

The current file size is **1870 bytes**.

`StudentFileHeader`

```
typedef struct {
    uint32_t studentProfileOffset;
    uint32_t academicRecordOffset;
    uint32_t hallDetailsOffset;
    uint32_t libraryDetailsOffset;
} StudentFileHeader;
```

Current size: **16 bytes**.

Offset	Size	Field	Current value
0	4	`studentProfileOffset`	16
4	4	`academicRecordOffset`	830
8	4	`hallDetailsOffset`	1478
12	4	`libraryDetailsOffset`	1590

Offsets are relative to the beginning of the `.student` file. A reader should use the header values rather than hard-coding them, although the current generator always produces the same values.

`StudentProfile`

```
typedef struct {
    char name[80];
    char department[50];
    char year[12];
    char term[12];
    char phone[16];
    char email[80];
    char bloodGroup[4];
    char fathersName[80];
    char mothersName[80];
    char permanentAddress[160];
    char currentAddress[160];
```

```
    char advisor[80];
} StudentProfile;
```

Current size: **814 bytes**. It begins at file offset 16.

Struct offset	Size	Field
0	80	name
80	50	department
130	12	year
142	12	term
154	16	phone
170	80	email
250	4	bloodGroup
254	80	fathersName
334	80	mothersName
414	160	permanentAddress
574	160	currentAddress
734	80	advisor

All fields in this struct are character arrays, so there is no internal alignment padding.

AcademicRecord

```
typedef struct {
    char name[80];
    int session;
    char departmentName[50];
    char departmentShortName[10];
    int year;
    int term;
    float creditTaken;
    float creditCompleted;
    float termGPA;
    float cgpa;
    Course courses[8];
} AcademicRecord;
```

Current size: **648 bytes**. It begins at file offset 830.

Struct offset	Size	Field
0	80	name
80	4	session
84	50	departmentName
134	10	departmentShortName

Struct offset	Size	Field
144	4	`year`
148	4	`term`
152	4	`creditTaken`
156	4	`creditCompleted`
160	4	`termGPA`
164	4	`cgpa`
168	480	`courses[8]`

There is no padding between `departmentShortName` and `year`: the first integer is already aligned because the preceding fields end at offset 144.

`Course`

Each `Course` is **60 bytes**. The eight courses occupy 480 bytes beginning at the `AcademicRecord` offset 168.

```
typedef struct {
    char courseCode[5];
    char courseShortName[12];
    char courseTitle[30];
    float gradePoint;
    float courseCredit;
    char letterGrade[3];
} Course;
```

Struct offset	Size	Field
0	5	`courseCode`
5	12	`courseShortName`
17	30	`courseTitle`
47	1	padding
48	4	`gradePoint`
52	4	`courseCredit`
56	3	`letterGrade`
59	1	trailing padding

Course `i` (zero-based) starts at:

```
course_offset = 168 + (i * 60)
```

The generator writes five theory courses followed by three laboratory courses. Unused course slots would remain zero-filled, but all eight slots are currently populated.

HallDetails

```
typedef struct {
    char hallName[50];
    int mealStatus;
    int boarderNumber;
    char hallSide[10];
    Date nextMonthlyFeast;
    int setMeal;
} HallDetails;
```

Current size: **112 bytes**. It begins at file offset 1478.

Struct offset	Size	Field
0	50	`hallName`
50	2	padding
52	4	`mealStatus`
56	4	`boarderNumber`
60	10	`hallSide`
70	2	padding
72	12	`nextMonthlyFeast`
84	4	`setMeal`
88	11	`offStartDate`
99	11	`offEndDate`

There are 2 trailing padding bytes after `offEndDate`, so the struct size is 112 bytes. The generator currently leaves both off-date fields empty.

The integer fields are written as native 4-byte integers.

LibraryDetails and IssuedBook

```
typedef struct {
    char bookTitle[80];
    Date returnDate;
} IssuedBook;

typedef struct {
    int bookCount;
    IssuedBook books[3];
} LibraryDetails;
```

`IssuedBook` is **92 bytes**:

Struct offset	Size	Field
0	80	`bookTitle`
80	12	`returnDate`

`LibraryDetails` is **280 bytes** and begins at file offset 1590:

Struct offset	Size	Field
0	4	`bookCount`
4	92	`books[0]`
96	92	`books[1]`
188	92	`books[2]`

Only the first `bookCount` entries are valid. The generator keeps `bookCount` between 0 and 3.

Complete `.student` layout

The following table is a convenient file-level map. The offsets come from the header and the current MinGW layout; a reader should still seek using the header values rather than assuming these constants.

File offset	Size	Record
0	16	`StudentFileHeader`
16	814	`StudentProfile`
830	648	`AcademicRecord`
1478	112	`HallDetails`
1590	280	`LibraryDetails`
1870	0	End of file

Reading and writing recommendations

Reading in C with the same layout

The safest option for the current format is to compile against the same struct definitions and read the complete record:

```
StudentFileHeader header;
StudentProfile profile;
AcademicRecord academic;
HallDetails hall;
LibraryDetails library;

fread(&header, sizeof(header), 1, fp);
```

```c
fseek(fp, (long) header.studentProfileOffset, SEEK_SET);
fread(&profile, sizeof(profile), 1, fp);
fseek(fp, (long) header.academicRecordOffset, SEEK_SET);
fread(&academic, sizeof(academic), 1, fp);
fseek(fp, (long) header.hallDetailsOffset, SEEK_SET);
fread(&hall, sizeof(hall), 1, fp);
fseek(fp, (long) header.libraryDetailsOffset, SEEK_SET);
fread(&library, sizeof(library), 1, fp);
```

Production code should check every `fread()` result and validate offsets against the file size before seeking.

Writing records

- Preserve the exact field widths and record order.
- For a `.student` file, write a 16-byte header first, then `StudentProfile`, `AcademicRecord`, `HallDetails`, and `LibraryDetails`.
- For a `.teacher` file, write exactly one 415-byte `TeacherProfile` record; there is no header and no trailing record.
- Zero-initialize structs before assigning fields.
- Do not write pointers, dynamically allocated strings, or `sizeof(pointer)`.
- Do not add, remove, or reorder fields without regenerating all existing files.
- Use `snprintf()` or equivalent bounded copying for fixed-size strings.
- A fixed-size string occupies its complete field width. Encode text as bytes, truncate to at most `field_width - 1` bytes, and append NUL bytes to fill the field. Do not write a length prefix.
- Keep `bookCount` in the range `0..3`.
- Keep `UserType` values within the documented enum values.
- Write integers as signed or unsigned 4-byte little-endian values as appropriate, and floats as 4-byte little-endian IEEE-754 values.
- Include every documented padding byte when constructing a file manually.

Cross-language readers

A Python, Java, Rust, or other non-C reader should parse each field manually using the documented offsets, fixed widths, little-endian integers, and little-endian IEEE-754 floats. It should not assume that a language's default struct unpacking matches C padding.

For example, these Python `struct` format fragments describe the current layout. The explicit `<` selects little-endian encoding and standard sizes; the `x` entries represent C alignment or trailing padding:

```python
import struct

HEADER = struct.Struct("<4I")                  # 16 bytes
COURSE = struct.Struct("<5s12s30sxff3sx")     # 60 bytes
ACADEMIC_PREFIX = struct.Struct("<80si50s10sii4f")  # 168 bytes
```

```
HALL = struct.Struct("<50s2xii10s2x12si11s11s2x")  # 112 bytes
ISSUED_BOOK = struct.Struct("<80s12s")       # 92 bytes
LIBRARY_HEADER = struct.Struct("<i")         # followed by 3 ISSUED_BOOK values
TEACHER = struct.Struct("<80s9s20s50s16s80s160s")  # 415 bytes
```

Write `bookCount` with `LIBRARY_HEADER` and then pack three `ISSUED_BOOK` values. Likewise, pack the academic prefix followed by eight `COURSE` values. A portable writer should assert the resulting lengths:

```
assert HEADER.size == 16
assert COURSE.size == 60
assert HALL.size == 112
assert TEACHER.size == 415
```

When creating a student file, calculate the four header offsets from the actual serialized block lengths:

```
profile_offset = HEADER.size
academic_offset = profile_offset + len(profile_bytes)
hall_offset = academic_offset + len(academic_bytes)
library_offset = hall_offset + len(hall_bytes)
header = HEADER.pack(profile_offset, academic_offset,
                     hall_offset, library_offset)
student_bytes = header + profile_bytes + academic_bytes + hall_bytes + library_bytes
```

The snippets are layout guidance, not a complete validation library. Check that every string fits, IDs match their filenames, offsets are in increasing order, and the final file length is exactly the expected end offset.

For long-term compatibility, consider changing the format to an explicitly versioned format with fixed-width integer types, defined endianness, and a serialization specification. The current format is suitable for tightly controlled builds but is not portable by itself.

Regenerating the files

From the directory containing `main.c`, compile and run:

```
gcc -std=c11 -Wall -Wextra -pedantic -o main.exe main.c
.\main.exe
```

Running the generator overwrites `credentials.dat`, all generated `.student` and `.teacher` files, and all generated `.notice` files. Generated non-default passwords are randomized on each run.